SBOM Supply Chain Risk Scoring: A Contextual Component Risk Ranking Framework

Final Research Report (Deliverable D4)

William A. Davis
M.S. Computer Science, Cybersecurity Engineering Track, Embry-Riddle Aeronautical University
CS 690 Graduate Research Project, Fall 2026
Advisor: Dr. Omar Ochoa
November 10, 2026

***Abstract*—Software Bills of Materials (SBOMs) tell an organization which third-party components its applications contain, but not which of their known vulnerabilities to fix first. Teams usually sort findings by CVSS severity, which describes how damaging a vulnerability could be, not how likely it is to be exploited or how exposed the affected component is. This project built and evaluated a rule-based, fully explainable pipeline that reads a CycloneDX SBOM, reconstructs the dependency graph, enriches every component with vulnerability data from OSV, NVD, FIRST EPSS and the CISA Known Exploited Vulnerabilities catalog, and ranks components by a weighted score of five factors: severity, exploit probability, known exploitation, dependency depth and version age. Each ranked component carries a one-sentence explanation of its position. Across six open-source applications in four ecosystems (3,109 components, 1,021 vulnerabilities, 310 vulnerable components ranked), the contextual ranking met the differentiation target set in advance in the three largest projects, every one of the 23 components that entered a top-10 list relative to the CVSS-only baseline was justified by a non-severity factor, and 107 of 120 weight-sensitivity runs stayed within the stability thresholds, with five of the six projects meeting the stability target.**

***Index Terms*—Software bill of materials, software supply chain security, vulnerability prioritization, CVSS, EPSS, dependency analysis**

I. Introduction

Modern applications are assembled largely from open-source components, and attacks on that supply chain have grown accordingly [1], [2]. In response, U.S. Executive Order 14028 directed federal suppliers to provide Software Bills of Materials [3], and the NTIA defined the minimum elements an SBOM must contain [4]. Generating an SBOM is now routine. Acting on one is not: a typical application has hundreds of components, and a vulnerability scan of its SBOM can return hundreds of findings with no indication of which matter most.

The default answer is to sort by the Common Vulnerability Scoring System (CVSS) base score [5]. CVSS measures the technical severity of a vulnerability in the abstract. It does not say whether anyone is exploiting it, and it knows nothing about where the vulnerable component sits in a particular application. Two vulnerabilities with a CVSS score of 9.8 can be very different in practice: one may be listed in CISA's catalog of actively exploited vulnerabilities in a library the application calls directly, while the other has almost no exploitation activity in a library four levels down.

This project asks whether adding exploitation evidence and application context to severity produces a ranking that is meaningfully different from CVSS alone, whether those differences can be justified, and whether the ranking is stable when the model's weights are changed. It makes four contributions:

1. A rule-based scoring model, defined in advance in a written specification (Deliverable D1), that combines five normalized factors and explains each component's rank in one sentence.

2. An open-source Python pipeline that implements the model end to end from a CycloneDX SBOM, with every external response cached so that results can be reproduced exactly.
3. A validation of the dependency graphs against each ecosystem's package manager, and of the vulnerability matches against independent tools, across npm, Python, Maven and Go.
4. An evaluation against a CVSS-only baseline on six applications, including a 20-run weight sensitivity analysis for each.

Section II introduces the data sources the model draws on. Section III reviews related work. Section IV defines the scoring model and the evaluation method, Section V describes the implementation and its validation, and Section VI reports the results. Section VII discusses what the results mean and where they are limited, Section VIII covers limitations and future work, and Section IX concludes.

II. Background

A. Software Bills of Materials

An SBOM is a machine-readable inventory of the components in a piece of software. The two dominant formats are CycloneDX [6], maintained by OWASP, and SPDX [7], maintained by the Linux Foundation. This project uses CycloneDX JSON. Each component carries a Package URL (purl) [8] such as pkg:pypi/django@4.1.7, which identifies the ecosystem, name and version unambiguously, and the document's dependencies section records which components depend on which. Together these allow the full dependency graph to be rebuilt from the SBOM alone.

B. Vulnerability Data

Vulnerabilities are identified by CVE numbers and catalogued in the National Vulnerability Database (NVD), which publishes CVSS base scores through its API [9]. The Open Source Vulnerabilities (OSV) database [10] aggregates advisories from ecosystem sources such as the GitHub Advisory Database and the Python Packaging Advisory Database and, importantly for this project, can be queried by package URL and exact version. OSV therefore answers the question "which vulnerabilities affect this component?", and NVD supplies the severity of each.

C. Exploitation Evidence

The Exploit Prediction Scoring System (EPSS) [11] estimates, for each CVE, the probability that it will be exploited in the wild within the next 30 days, using a model trained on observed exploitation activity. FIRST publishes daily scores through a public API [12]. The CISA Known Exploited Vulnerabilities (KEV) catalog [13] lists CVEs for which there is reliable evidence of active exploitation. EPSS is a forward-looking probability and KEV is a record of fact; the model uses both.

III. Related Work

Studies of open-source ecosystems show why dependency structure matters. Decan et al. [14] found that vulnerable npm packages are frequently reached through transitive dependencies and that fixes often take a long time to propagate through the dependency network. Zimmermann et al. [15] showed that a small number of npm maintainers and packages can affect a large share of the ecosystem. Kula et al. [16] found that developers frequently leave outdated library versions in place even after vulnerabilities are disclosed. Ohm et al. [1] and Ladisa et al. [2] catalogue the attacks that exploit this structure.

Several lines of work question severity-only prioritization. Allodi and Massacci [17] showed that a high CVSS score is a weak predictor of whether a vulnerability is actually exploited. EPSS [11] was developed to fill that gap with a data-driven probability. The Stakeholder-Specific Vulnerability Categorization (SSVC) [18] replaces a single score with a decision tree that considers exploitation status and the system's exposure. This project shares SSVC's

premise that context matters, but keeps a numeric score so that components can be ranked and compared directly with CVSS.

Research on SBOMs has focused mainly on their generation, quality and adoption. Xia et al. [19] and Stalnaker et al. [20] surveyed practitioners and found that SBOM consumption, including turning an SBOM into security decisions, lags well behind generation. Ponta et al. [21] showed that code-level reachability analysis can tell whether a vulnerable function is actually used, which is more precise than any metadata-based measure but requires the application's source code and language-specific tooling. The present work sits between these: it uses only the SBOM and public data, so it applies to any application with an SBOM, and it uses dependency depth as an inexpensive structural proxy for exposure rather than true reachability.

IV. Methodology

The project followed a build-and-evaluate method. The scoring model was specified in writing and approved by the advisor before any scoring code was written, so that the evaluation could not be tuned to the results.

A. Research Questions

- **RQ1, differentiation:** Does the contextual ranking differ meaningfully from a CVSS-only ranking of the same components?
- **RQ2, justification:** When a component rises into the top 10 relative to the baseline, is the cause a factor that CVSS does not capture?
- **RQ3, stability:** Does the ranking remain substantially the same when individual weights are varied within a reasonable range?

B. Scoring Model

Only components with at least one known vulnerability are ranked. Each factor is scaled to the range 0 to 1, as shown in Table I.

TABLE I. SCORING FACTORS

Factor	Definition	When data is missing
S, Severity	CVSS base score divided by 10, preferring v3.1, then v4.0, v3.0 and v2	The advisory's qualitative rating (Critical 0.95, High 0.80, Moderate 0.55, Low 0.25), else 0.50, flagged
E, Exploit probability	EPSS probability on the analysis date	0, flagged (EPSS covers CVEs only)
X, Known exploitation	1.0 if in CISA KEV; 0.5 if NVD lists public exploit code; else 0	0
D, Dependency depth	1 divided by the shortest depth from the application (direct dependency = 1.0)	Maximum depth plus one, flagged
A, Component age	Days since the version's release divided by 1,826, capped at 1.0	0.50, flagged

Each vulnerability v receives a score from its own three factors, and a component c is scored on its worst vulnerability plus its two component-level factors:

$$VulnScore(v) = w_1 S(v) + w_2 E(v) + w_3 X(v)$$

$$Risk(c) = \max_{v \in c} VulnScore(v) + w_4 D(c) + w_5 A(c)$$

The default weights are $w_1 = 0.30$, $w_2 = 0.25$, $w_3 = 0.20$, $w_4 = 0.15$ and $w_5 = 0.10$. They sum to 1, so the risk score also lies between 0 and 1 and is reported multiplied by 100. Scoring a component on its single worst vulnerability, rather than on the worst value of each factor separately, prevents the model from combining one CVE's severity with another CVE's exploit probability into a vulnerability that does not exist. The vulnerability that produces the maximum is called the driving vulnerability. A component is scored on its worst vulnerability rather than on the sum of all of them for two reasons. Upgrading a component usually removes all of its vulnerabilities at once, so the most urgent one determines how urgent that single fix is. A sum would also rank large, heavily scrutinized libraries above smaller ones simply because more advisories have been filed against them. The number of vulnerabilities is not ignored: it is the first tie-breaker when two components have the same score.

The weights were set by judgment and documented with their rationale in D1. Severity keeps the largest weight because it remains the most informative signal when nothing else is known; the two exploitation factors together carry 45 percent because they measure what CVSS does not; depth and age are contextual adjustments and together carry 25 percent. Because these values are not learned from data, the evaluation tests how much the ranking depends on them (RQ3).

Every ranked component receives a one-sentence explanation naming its driving vulnerability, its two largest weighted contributions with their point values, its depth and dependency path, its version's age, any data-quality flags, and its rank in the baseline.

C. Baseline

The CVSS-only baseline orders the same components by the highest severity among their vulnerabilities, using the same fallbacks as factor S, with ties broken by the number of vulnerabilities and then by name. Because both rankings are computed from identical data, any difference between them is caused by the four non-severity factors alone.

D. Test Projects

Four applications were first selected, one per major ecosystem, each pinned to a release tag (Table II). OWASP Juice Shop [22], PyGoat [23] and WebGoat [24] are deliberately vulnerable training applications, which guarantees that there are vulnerable components to rank. Because such applications might overstate how often context changes a ranking, Gitea [25], a production Git service, was included as a check on that bias. The four ecosystems also differ structurally: npm and Maven trees are deep, while Go module graphs are shallow by design. After the first evaluation showed that two of these projects had too few vulnerable components to test differentiation fairly (Section VI-A), two larger production applications were added at the advisor's request on November 10, 2026: Etherpad 1.8.6, a collaborative editor, and Wiki.js 2.5.300, a wiki platform, both npm projects. Both were added before their results were known, and all six projects are reported.

E. Evaluation Measures

The measures and their targets were fixed in D1 before evaluation:

- **Differentiation (RQ1):** the number of components in the contextual top 10 that are not in the baseline top 10, and Kendall's τ [26] between the two full rankings. Target: at least 3 of 10 different in at least three projects. D1 stated this as three of the four original projects; when two projects were added, the required count of three was kept unchanged. For a project with ten or fewer vulnerable components the two top-10 lists necessarily coincide, so τ alone is used.
- **Justification (RQ2):** for every component entering the top 10 relative to the baseline, one of its two largest weighted contributions must be exploit probability, known exploitation or depth. Target: all entering components.

- **Weight stability (RQ3):** each of the five weights is changed by −0.10, −0.05, +0.05 and +0.10, with the other four rescaled proportionally to keep the sum at 1, giving 20 runs per project. Each run is compared with the default ranking. Target: a top-10 overlap of at least 8 and τ of at least 0.80 in 90 percent of runs.
- **Vulnerability density:** the number of vulnerable components at each depth and the share of total risk held by direct dependencies, reported without a target.

V. Implementation

A. Pipeline

The pipeline is a Python 3 command-line tool, sbomrisk, that uses only the standard library. It has five stages, each a separate module: parsing the CycloneDX document; rebuilding the dependency graph and computing each component's shortest depth from the application by breadth-first search; enrichment; scoring and ranking; and evaluation. A typical run is three commands:

```
sbomrisk enrich data\sboms\pygoat-v2.0.1.cdx.json
sbomrisk rank data\sboms\pygoat-v2.0.1.cdx.json
sbomrisk evaluate
```

Enrichment queries OSV in batches by package URL, merges records that are aliases of one another (for example a GitHub advisory and its CVE) into one vulnerability, drops withdrawn records, and then retrieves CVSS from NVD, EPSS probabilities in batches of 100, the KEV catalog, and each vulnerable version's release date from its package registry (npm, PyPI, the Go module proxy or Maven Central). When npm installs the same package version in several places, the copies are merged into one entry at the shallowest depth. Requests are paced to respect NVD's rate limits and retried with exponential backoff.

Every API response is stored on disk, and each run records its analysis date, the weights, a SHA-256 hash of the input SBOM and the count of every data-quality flag. A run can therefore be repeated exactly, offline, from its cache. The weights are read from a configuration file, so the sensitivity analysis required no code changes. The ranking is written as JSON and as CSV for spreadsheet review, together with the baseline ranking and a list of components that were not ranked and why.

B. SBOM Generation

The proposal planned to generate SBOMs with Syft [27]. When tested against the original four projects' source trees, Syft found the packages but recorded almost none of the relationships between them: zero dependency links for Juice Shop and PyGoat. Without relationships, depth cannot be computed. The SBOMs were therefore generated with each ecosystem's CycloneDX generator [28], which resolves the full graph, and every generation command was recorded with pinned tool versions in a script so the SBOMs can be regenerated.

TABLE II. TEST PROJECTS

Project	Release	Ecosystem	Components	Dependency links	Max depth
OWASP Juice Shop	v20.2.0	npm	689	1,063	7
OWASP PyGoat	v2.0.1	Python	33	41	3
OWASP WebGoat	v2026.4	Maven	191	380	8
Gitea	v1.21.0	Go	231	733	3
Etherpad	1.8.6	npm	953	1,554	8
Wiki.js	v2.5.300	npm	1,367	2,441	11

C. Validation

Because depth is a scoring factor, the reconstructed graphs were validated against each ecosystem's own package manager on the author's workstation. The parser matched npm ls on all 645 installed Juice Shop packages and their depths, pipdeptree on all 33 PyGoat packages and 25 required links, go list on all 231 Gitea modules and versions, Maven's dependency tree on all 179 WebGoat runtime libraries and their depths, and npm ls on 789 of 790 Etherpad packages (one differs only in the capitalization of its name) and all 1,216 Wiki.js packages, each at the same depth. The validation also exposed a genuine error in the Python SBOM generator, which recorded ten links from optional extras as required dependencies and placed three packages at the wrong depth; the generation script now removes links that pip does not confirm.

Vulnerability matching was checked against independent tools that use the same advisory sources through a different path: PyPI's own vulnerability data, which pip-audit [29] uses, for PyGoat, and the npm registry's advisory service, which npm audit [30] uses, for Juice Shop. Section VI-E reports the agreement.

D. Testing

The project has 36 automated tests. They exercise a hand-built SBOM containing cycles, duplicate paths and unreachable components; enrichment against recorded API responses, including alias merging, withdrawn records, missing scores and offline replay; and scoring, including both worked examples from D1, which the implementation reproduces exactly (94.9 and 34.2), every missing-data fallback, the tie-breaking rules, and the weight variation used in the sensitivity analysis.

VI. Results

All six projects were enriched and ranked without error. The original four were run on November 6, 2026 and the two added projects on November 10, 2026, each with its own recorded analysis date. Together they contain 3,109 components after duplicate installations are merged, of which 310 have at least one known vulnerability, with 1,021 vulnerabilities in total. Table III summarizes the comparison with the baseline.

TABLE III. DIFFERENTIATION FROM THE CVSS-ONLY BASELINE

Project	Ecosystem	Components	Ranked	Vulnerabilities	New in top 10	Kendall's τ	Rank changes
Juice Shop	npm	645	34	93	5 of 10	0.39	31
PyGoat	Python	33	14	145	1 of 10	0.58	13
WebGoat	Maven	190	8	56	n/a	0.64	7
Gitea	Go	231	26	105	2 of 10	0.68	24
Etherpad	npm	794	86	216	7 of 10	0.40	82
Wiki.js	npm	1,216	142	406	8 of 10	0.47	138

A. RQ1: Differentiation

The differentiation target was met: at least three of the contextual top 10 were new relative to the baseline in three projects, Wiki.js (8), Etherpad (7) and Juice Shop (5). These are the three largest projects, with 142, 86 and 34 vulnerable components. In the original four-project evaluation the target was met in Juice Shop only, and the reason was size rather than the model. WebGoat has only eight vulnerable components, so its top 10 necessarily contains all of them and only τ applies; D1 anticipated this case. PyGoat, with 14 vulnerable components, can admit at most four

new entries and admitted one (idna). Gitea, with 26, admitted two. The two added projects test the question at the scale where it matters, and both cleared it comfortably. The order changed substantially everywhere: τ ranged from 0.39 to 0.68, and between 88 and 97 percent of components changed position in each project. In Juice Shop, notevil 1.3.3 rose from #27 to #4 because it is a direct dependency with public exploit code despite a CVSS score of 6.5; in Wiki.js, validate.js 0.13.1 rose from #94 to #6 as a direct dependency; and in both added projects y18n rose into the top three on an EPSS probability of 0.70. The weights were not adjusted at any point.

B. RQ2: Justification

Table IV lists every component that entered a top 10 relative to the baseline. In all 23 cases, one of its two largest weighted contributions was a factor that CVSS does not capture. Thirteen entries are explained by depth, seven by exploit evidence (public exploit code listed in NVD) and three by EPSS. Severity is always the other leading factor, which is by design: context can promote a component, but not one without a significant vulnerability. In the original four projects no entry was explained by EPSS, because the high EPSS values there belonged to components already in the baseline top 10, such as Pillow in PyGoat and brotli and golang.org/x/crypto in Gitea. In the two larger projects, y18n (EPSS 0.70) entered the top 10 from baseline ranks of #14 to #19 on its exploit probability, showing that EPSS does move components across the boundary when the list is long enough.

TABLE IV. COMPONENTS ENTERING THE TOP 10

Project	Component	Baseline → contextual	Two largest factors	Justified
Juice Shop	notevil@1.3.3	#27 → #4	severity, depth	Yes
Juice Shop	sanitize-html@1.4.2	#14 → #5	severity, depth	Yes
Juice Shop	uuid@8.3.2	#23 → #8	severity, exploit evidence	Yes
Juice Shop	minimatch@3.0.5	#15 → #9	severity, exploit evidence	Yes
Juice Shop	ws@7.4.6	#19 → #10	severity, exploit evidence	Yes
PyGoat	idna@3.4	#11 → #6	severity, exploit evidence	Yes
Gitea	github.com/golang-jwt/jwt/ v5@v5.0.0	#14 → #9	severity, depth	Yes
Gitea	google.golang.org/ protobuf@v1.31.0	#16 → #10	severity, depth	Yes
Etherpad	y18n@4.0.0	#15 → #2	severity, EPSS	Yes
Etherpad	y18n@3.2.1	#14 → #3	severity, EPSS	Yes
Etherpad	async@3.2.0	#25 → #4	severity, depth	Yes
Etherpad	terser@4.7.0	#58 → #5	severity, depth	Yes
Etherpad	semver@5.6.0	#55 → #6	severity, depth	Yes
Etherpad	underscore@1.8.3	#64 → #9	severity, depth	Yes
Etherpad	tough-cookie@2.5.0	#13 → #10	severity, exploit evidence	Yes
Wiki.js	y18n@4.0.0	#19 → #1	severity, EPSS	Yes
Wiki.js	sqlite3@5.1.4	#15 → #2	severity, depth	Yes
Wiki.js	knex@0.21.7	#74 → #4	severity, depth	Yes
Wiki.js	js-yaml@3.14.0	#41 → #5	severity, depth	Yes
Wiki.js	validate.js@0.13.1	#94 → #6	severity, depth	Yes
Wiki.js	image-size@0.9.2	#71 → #7	severity, depth	Yes

Project	Component	Baseline → contextual	Two largest factors	Justified
Wiki.js	tough-cookie@2.5.0	#16 → #8	severity, exploit evidence	Yes
Wiki.js	tough-cookie@3.0.1	#17 → #9	severity, exploit evidence	Yes

C. RQ3: Weight Stability

107 of the 120 sensitivity runs met both stability thresholds, and five of the six projects met the 90 percent threshold (Gitea 20 of 20, PyGoat 19, Etherpad 19, Juice Shop 18, WebGoat 18). Wiki.js did not: 13 of its 20 runs were stable, so the stability target as defined in D1 was not met for every project. In the original four projects, all five runs outside the thresholds came from the largest changes (±0.10). WebGoat's two reflect its size: with eight components, two adjacent swaps lower τ to 0.64. Juice Shop's two both displace uuid, minimatch and ws, the three components that entered its top 10 on exploit evidence. Etherpad's one comes from lowering the depth weight by 0.10. Wiki.js's seven come from all four depth variations, both ±0.10 severity variations and the −0.10 exploit-evidence variation. The cause is how tightly its scores are packed: after the top three, places 4 through 12 lie within 1.5 points of each other, five components are tied at 57.6, and 13 components lie within two points of tenth place, about twice as many as in any other project. Small weight changes therefore swap components across the tenth-place boundary, and every failing run fails on top-10 overlap (6 or 7 of 10). The full ranking is far less affected: τ against the default ranking stayed above 0.76 in every Wiki.js run, and only two runs fell below 0.80. The top-10 overlap measure is thus most fragile in exactly the large projects where the model changes the most, which is a limitation of the measure as well as a property of the ranking.

TABLE V. WEIGHT SENSITIVITY (20 RUNS PER PROJECT)

Project	Stable runs	Lowest top-10 overlap	Lowest τ	Variations outside the thresholds
Juice Shop	18 of 20	7	0.78	exploit evidence −0.10, age −0.10
PyGoat	19 of 20	7	0.76	depth +0.10
WebGoat	18 of 20	8	0.64	depth −0.10, depth +0.10
Gitea	20 of 20	9	0.85	none
Etherpad	19 of 20	6	0.85	depth −0.10
Wiki.js	13 of 20	6	0.77	depth ±0.05 and ±0.10, severity ±0.10, exploit evidence −0.10

D. Vulnerability Density

TABLE VI. VULNERABLE COMPONENTS BY DEPTH

Project	Vulnerable components by depth	Share that are direct	Share of total risk held by direct dependencies
Juice Shop	1: 11, 2: 15, 3: 5, 4: 1, 5: 1, 6: 1	32 percent	36 percent
PyGoat	1: 9, 2: 4, 3: 1	64 percent	70 percent
WebGoat	1: 4, 2: 2, 3: 1, 4: 1	50 percent	61 percent
Gitea	1: 19, 2: 7	73 percent	74 percent
Etherpad	1: 10, 2: 31, 3: 18, 4: 14, 5: 4, 6: 8, 7: 1	12 percent	13 percent
Wiki.js	1: 34, 2: 50, 3: 26, 4: 14, 5: 14, 6: 2, 7: 2	24 percent	26 percent

In every project a direct dependency is much more likely to carry a known vulnerability than a deep one. In Juice Shop, 11 of 63 direct dependencies (17 percent) are vulnerable, against 15 of 178 at depth two (8 percent), 5 of 179 at depth three (3 percent) and 3 of 269 below that (about 1 percent). Gitea shows the same pattern (17 percent of direct dependencies against 6 percent at depth two), as do WebGoat (11 percent against 5, 3 and 1 percent) and PyGoat, whose small and old dependency set is vulnerable at every level (56 percent of direct dependencies). The two added projects follow the same pattern: in Wiki.js 22 percent of direct dependencies are vulnerable against 8 percent four or more levels down, and in Etherpad 25 percent against 9 percent.

Direct dependencies hold most of the total risk score in Gitea (74 percent), PyGoat (70 percent) and WebGoat (61 percent). Juice Shop is the exception at 36 percent, because several of its most dangerous components, such as jsonwebtoken 0.1.0 (through express-jwt) and lodash 2.4.2 (through sanitize-html), are pulled in transitively by old direct dependencies. That pattern is the case the depth factor is meant to temper rather than ignore: these components still rank #2 and #3. Etherpad (13 percent) and Wiki.js (26 percent) are lower still, for the same reason: in large npm graphs most vulnerable components arrive through a small number of old direct dependencies.

E. Agreement With Independent Tools

For PyGoat, PyPI reports 141 distinct vulnerabilities across the 33 components, and all 141 are among the 145 the pipeline found. The four additional findings (two in Pillow and two in oauthlib) are recent OSV records not yet mirrored to PyPI. Per-component counts agree for 12 of the 14 vulnerable components, and the 19 components the pipeline reports as clean have no PyPI records either.

For Juice Shop, the npm registry reports 77 advisories across 30 packages, and all 77 are among the pipeline's findings. Of the 12 additional findings, 11 concern multer 1.4.5-lts.2. The npm advisories' own ranges cover that version, but npm's version matching [31] skips versions with a suffix such as "-lts.2", so npm audit reports multer as clean. OSV matches it, and the pipeline ranks multer #18 with 11 advisories. The twelfth, for messageformat 2.3.0, appears to be an OSV over-report: npm gives the affected range as below 2.3.0.

F. Example Rankings

TABLE VII. JUICE SHOP: CONTEXTUAL TOP 10

Rank	Component	Score	CVSS-only rank	Driving vulnerability (CVSS)	Largest factors
1	jsonwebtoken 0.4.0	66.5	2	CVE-2015-9235 (9.8)	severity, depth
2	jsonwebtoken 0.1.0	59.0	1	CVE-2015-9235 (9.8)	severity, exploit evidence
3	lodash 2.4.2	56.1	7	CVE-2019-10744 (9.1)	severity, exploit evidence
4	notevil 1.3.3	54.8	27	CVE-2021-23771 (6.5)	severity, depth
5	sanitize-html 1.4.2	53.6	14	CVE-2017-16016 (6.1)	severity, depth
6	marsdb 0.6.11	53.5	4	GHSA-5mrr-rgp6-x4gr (n/a)	severity, depth
7	express-jwt 0.1.3	52.6	9	CVE-2020-15084 (9.1)	severity, depth
8	uuid 8.3.2	50.1	23	CVE-2026-41907 (7.5)	severity, exploit evidence
9	minimatch 3.0.5	49.6	15	CVE-2026-26996 (7.5)	severity, exploit evidence
10	ws 7.4.6	47.9	19	CVE-2026-48779 (7.5)	severity, exploit

Rank	Component	Score	CVSS-only rank	Driving vulnerability (CVSS)	Largest factors
					evidence

TABLE VIII. PYGOAT: CONTEXTUAL TOP 10

Rank	Component	Score	CVSS-only rank	Driving vulnerability (CVSS)	Largest factors
1	pillow 9.4.0	94.1	1	CVE-2023-4863 (8.8)	severity, EPSS
2	pyyaml 5.1	65.7	4	CVE-2019-20477 (9.8)	severity, depth
3	django 4.1.7	64.6	2	CVE-2025-64459 (9.1)	severity, depth
4	pyjwt 2.4.0	61.3	5	CVE-2026-102268 (9.1)	severity, depth
5	cryptography 39.0.1	55.2	6	CVE-2023-49083 (7.5)	severity, depth
6	idna 3.4	48.6	11	CVE-2024-3651 (7.5)	severity, exploit evidence
7	werkzeug 2.1.2	47.4	10	CVE-2024-34069 (7.5)	severity, depth
8	urllib3 1.26.9	46.6	9	CVE-2024-37891 (6.5)	severity, exploit evidence
9	certifi 2022.12.7	44.9	3	CVE-2023-37920 (9.8)	severity, age
10	sqlparse 0.3.1	43.7	7	CVE-2026-54284 (8.7)	severity, age

TABLE IX. WEBGOAT: CONTEXTUAL TOP 10

Rank	Component	Score	CVSS-only rank	Driving vulnerability (CVSS)	Largest factors
1	com.thoughtworks.xstream:xstream 1.4.5	95.0	1	CVE-2021-39144 (8.5)	severity, EPSS
2	com.google.guava:guava 33.7.1-jre	44.7	3	CVE-2026-102554 (8.2)	severity, depth
3	com.fasterxml.jackson.core:jackson-databind 2.21.5	42.7	4	CVE-2026-68497 (7.5)	severity, depth
4	org.apache.tomcat.embed:tomcat-embed-core 11.0.24	38.3	2	CVE-2026-65905 (9.8)	severity, age
5	org.apache.commons:commons-lang3 3.14.0	37.5	8	CVE-2025-48924 (5.3)	severity, depth
6	tools.jackson.core:jackson-databind 3.1.5	35.2	5	CVE-2026-68497 (7.5)	severity, depth
7	com.fasterxml.jackson.core:jackson-core 2.21.5	35.2	6	CVE-2026-89407 (7.5)	severity, depth
8	tools.jackson.core:jackson-core 3.1.5	32.7	7	CVE-2026-89407 (7.5)	severity, age

TABLE X. GITEA: CONTEXTUAL TOP 10

Rank	Component	Score	CVSS-only rank	Driving vulnerability (CVSS)	Largest factors
1	github.com/andybalholm/	85.8	4	CVE-2026-111497 (9.8)	severity, EPSS

Rank	Component	Score	CVSS-only rank	Driving vulnerability (CVSS)	Largest factors
	brotli v1.0.5				
2	golang.org/x/crypto v0.14.0	72.3	1	CVE-2023-48795 (5.9)	EPSS, severity
3	golang.org/x/net v0.17.0	66.0	5	CVE-2023-45288 (7.5)	severity, EPSS
4	github.com/go-git/go-git/v5 v5.9.0	51.1	2	CVE-2023-49569 (9.8)	severity, depth
5	github.com/mholt/archiver/v3 v3.5.1	49.4	10	CVE-2025-3445 (8.1)	severity, depth
6	google.golang.org/grpc v1.58.3	48.9	6	CVE-2026-33186 (9.1)	severity, depth
7	github.com/klauspost/ compress v1.17.0	47.5	7	CVE-2026-116591 (8.7)	severity, depth
8	github.com/go-git/go-billy/v5 v5.5.0	45.7	9	CVE-2026-44973 (8.1)	severity, depth
9	github.com/golang-jwt/jwt/v5 v5.0.0	44.8	14	CVE-2025-30204 (7.5)	severity, depth
10	google.golang.org/protobuf v1.31.0	44.5	16	CVE-2024-24786 (7.5)	severity, depth

VII. Discussion

The results support the project's central claim in a qualified form. A contextual score does reorder vulnerability work, in every project and for reasons that can be stated in one sentence per component. Where the reordering crosses the top-10 boundary, the cause is always evidence that CVSS does not capture: exploit probability, public exploit code or a direct dependency. The ordering is also largely stable under reasonable changes to the weights: five of the six projects met the stability target, and in the sixth, Wiki.js, the changes were confined to near-tied components at the tenth-place boundary. The result is therefore not an artifact of the particular weights chosen, although in very large projects the exact membership of the top 10 does depend on them.

The results also show where the effect is concentrated. In small or old dependency sets, the components with the highest CVSS scores also tend to be direct, old and exploited, so the factors agree and the same components lead both lists in a different order. In the three large projects (Juice Shop, Etherpad and Wiki.js, with 34 to 142 vulnerable components) context changed between five and eight of the top 10. Contextual scoring therefore matters most where dependency trees are large, which is where triage is hardest and where a CVSS-only sort produces the longest undifferentiated list. The same scale has a cost: in Wiki.js many components score within a point or two of each other, so the exact membership of a top 10 is sensitive to the weights even though the overall order is not. A practical reading is that, in very large projects, the model is best used to identify a band of top candidates rather than a precise tenth place.

Two practical findings emerged from the cross-checks. Matching through OSV found every vulnerability that pip-audit's and npm audit's sources report, and it caught 11 multer advisories that npm audit misses because of how npm compares versions. The evaluation also exposed a data gap: the Maven Central search index did not return recent releases, so six WebGoat components fell back to the neutral age value. Supplying the dates from the repository itself changed WebGoat's order slightly (τ against the baseline 0.50 instead of 0.64) and changed no conclusion. Finally, PyGoat's data changed between the October 23 and November 6 runs (two new advisories, one

CVSS score raised from 9.2 to 9.8), which is why every result is tied to a recorded analysis date and cached responses.

A. Threats to Validity

- **No ground truth.** There is no labeled data set of which component a security team should fix first, so the evaluation measures whether the ranking differs from CVSS for reasons that can be stated, not whether it is correct. The justification measure and the per-component explanations are the project's answer to this, but a user study with practitioners would be the stronger test.
- **Weights set by judgment.** The weights were documented in advance and the sensitivity analysis shows how much the ranking depends on them, but they were not calibrated against outcomes. In the largest project, Wiki.js, the top-10 boundary was sensitive to the weights because of near-ties.
- **Training applications.** Three of the six projects are intentionally vulnerable, which may exaggerate how often context changes the order. Gitea, Etherpad and Wiki.js are production applications and were included to check this; the largest effects appeared in two of them.
- **Upstream data quality.** OSV and the advisory databases it aggregates sometimes list a vulnerability for versions outside the range stated in the CVE description. For example, CVE-2026-69249 is listed for cryptography 39.0.1 by both OSV and PyPI, although NVD's description names versions 42.0.0 through 48.0.0. The pipeline reports what the databases return.
- **Point in time.** EPSS scores change daily and new advisories appear constantly. Each project's results are tied to its recorded analysis date and cached responses.
- **SBOM correctness.** The pipeline can only be as accurate as its input. This is why the graphs were validated against each package manager before scoring.

VIII. Limitations and Future Work

Depth is a structural measure: it shows how close a library sits to the application, not whether the vulnerable function is ever called. Combining the score with code-level reachability analysis [21] for ecosystems where it is available is the most valuable extension. EPSS covers only CVE identifiers, so advisories without a CVE receive an exploit probability of zero and are flagged. Go module graphs are shallow, which makes the depth factor less discriminating for Go. SPDX input was out of scope and is a natural next step, since the parser's output model does not depend on the input format. Finally, a study in which security practitioners rank the same components would test whether the contextual order matches expert judgment. The stability measure itself could be improved: a top-10 overlap count treats a swap between tenth and eleventh place, separated by a fraction of a point, the same as a large change, so a tie-aware measure would describe very large projects more fairly. In addition, because each component is scored on its single worst vulnerability, a component with several known-exploited vulnerabilities scores the same on the exploitation factors as one with a single such vulnerability; a small bonus for multiple exploited vulnerabilities could be tested.

IX. Conclusion

This project built a rule-based pipeline that turns a CycloneDX SBOM into a ranked, explained list of vulnerable components, using only the SBOM and public data. Evaluated on six applications in four ecosystems, the contextual ranking differed from a CVSS-only ranking in every project (τ 0.39 to 0.68), and it changed at least three of the top 10 in the three largest projects, meeting the differentiation target. Every one of the 23 components it promoted into a top 10 was promoted for a stated non-severity reason, and 107 of 120 weight variations left the ranking substantially unchanged. Five of the six projects met the stability target; in the largest, Wiki.js, near-ties at the tenth-place

boundary made the top-10 membership sensitive to the weights while the overall order held. Contextual scoring adds the most where dependency graphs are large.

The pipeline, its tests, the evaluation data and the per-component explanations are published so that each result can be reproduced and checked. The most valuable next steps are code-level reachability for the depth factor, a fallback source for Maven release dates, and a study with practitioners to test whether the contextual order matches the order in which experts would fix the same components.

References

[1] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks," in Proc. Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA), 2020, pp. 23–43.

[2] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains," in Proc. IEEE Symp. Security and Privacy (SP), 2023, pp. 1509–1526.

[3] The White House, "Executive Order 14028: Improving the Nation's Cybersecurity," Federal Register, vol. 86, no. 93, pp. 26633–26647, May 2021.

[4] National Telecommunications and Information Administration, "The Minimum Elements for a Software Bill of Materials (SBOM)," U.S. Department of Commerce, Jul. 2021.

[5] FIRST, "Common Vulnerability Scoring System v3.1: Specification Document," Jun. 2019. [Online]. Available: https://www.first.org/cvss/v3.1/specification-document

[6] OWASP Foundation, "CycloneDX Specification, Version 1.5," 2023. [Online]. Available: https://cyclonedx.org/docs/1.5/json/

[7] The Linux Foundation, "SPDX Specification, Version 2.3," 2022. [Online]. Available: https://spdx.dev

[8] Package URL Project, "Package URL (purl) Specification." [Online]. Available: https://github.com/package-url/purl-spec

[9] NIST, "National Vulnerability Database API v2.0." [Online]. Available: https://nvd.nist.gov/developers/vulnerabilities

[10] Google and OpenSSF, "Open Source Vulnerabilities (OSV) Database and API." [Online]. Available: https://osv.dev

[11] J. Jacobs, S. Romanosky, B. Edwards, I. Adjerid, and M. Roytman, "Exploit Prediction Scoring System (EPSS)," Digital Threats: Research and Practice, vol. 2, no. 3, pp. 1–17, 2021.

[12] FIRST, "Exploit Prediction Scoring System (EPSS) API." [Online]. Available: https://www.first.org/epss/api

[13] CISA, "Known Exploited Vulnerabilities Catalog." [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[14] A. Decan, T. Mens, and E. Constantinou, "On the Impact of Security Vulnerabilities in the npm Package Dependency Network," in Proc. 15th Int. Conf. Mining Software Repositories (MSR), 2018, pp. 181–191.

[15] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small World with High Risks: A Study of Security Threats in the npm Ecosystem," in Proc. 28th USENIX Security Symp., 2019, pp. 995–1010.

[16] R. G. Kula, D. M. German, A. Ouni, T. Ishio, and K. Inoue, "Do Developers Update Their Library Dependencies?," Empirical Software Engineering, vol. 23, no. 1, pp. 384–417, 2018.

[17] L. Allodi and F. Massacci, "Comparing Vulnerability Severity and Exploits Using Case-Control Studies," ACM Trans. Information and System Security, vol. 17, no. 1, pp. 1–20, 2014.

[18] J. Spring, E. Hatleback, A. Householder, A. Manion, and D. Shick, "Prioritizing Vulnerability Response: A Stakeholder-Specific Vulnerability Categorization," Carnegie Mellon Univ. Software Engineering Institute, Tech. Rep., 2019.

[19] B. Xia, T. Bi, Z. Xing, Q. Lu, and L. Zhu, "An Empirical Study on Software Bill of Materials: Where We Stand and the Road Ahead," in Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE), 2023, pp. 2630–2642.

[20] T. Stalnaker, N. Wintersgill, O. Chaparro, M. Di Penta, D. M. German, and D. Poshyvanyk, "BOMs Away! Inside the Minds of Stakeholders: A Comprehensive Study of Bills of Materials for Software Systems," in Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE), 2024.

[21] S. E. Ponta, H. Plate, and A. Sabetta, "Beyond Metadata: Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software," in Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME), 2018, pp. 449–460.

[22] OWASP Foundation, "OWASP Juice Shop," release v20.2.0. [Online]. Available: https://github.com/juice-shop/juice-shop

[23] OWASP Foundation, "OWASP PyGoat," release v2.0.1. [Online]. Available: https://github.com/adeyosemanputra/pygoat

[24] OWASP Foundation, "OWASP WebGoat," release v2026.4. [Online]. Available: https://github.com/WebGoat/WebGoat

[25] Gitea Authors, "Gitea," release v1.21.0. [Online]. Available: https://github.com/go-gitea/gitea

[26] M. G. Kendall, "A New Measure of Rank Correlation," Biometrika, vol. 30, no. 1/2, pp. 81–93, 1938.

[27] Anchore, "Syft." [Online]. Available: https://github.com/anchore/syft

[28] CycloneDX, "CycloneDX Tool Center." [Online]. Available: https://cyclonedx.org/tool-center/

[29] Python Packaging Authority, "pip-audit." [Online]. Available: https://github.com/pypa/pip-audit

[30] npm, Inc., "npm audit," npm Docs. [Online]. Available: https://docs.npmjs.com/cli/commands/npm-audit

[31] npm, Inc., "node-semver: The Semantic Versioner for npm." [Online]. Available: https://github.com/npm/node-semver